

Prompt de recherche M4 — avalanches auto-organisées

Version pour Claude Fable 5 (workspace `m4_credit_soc_fable`)

Anatole Joseph Stouls

13 juillet 2026

Résumé

Fork de M3 (`m3_credit_soc`) : société de crédit multi-agents en recherche d'un régime d'avalanches de faillites auto-organisé (SOC), robuste en loi de puissance, dépassant nettement les tailles obtenues jusqu'ici ($\max = 7$), avec un code aussi simple que possible. Ce document est le brief de recherche remis à Claude Fable 5 pour cette phase ; les instructions d'exécution propres au modèle sont en fin de document.

Lecture préalable obligatoire

- `m3_credit_soc/NOTES.md` — journal complet du programme M3. Utile comme historique de ce qui a déjà été essayé, **pas comme verdict à respecter** (voir plus bas). Ne relancez pas un test déjà fait sans le citer et dire pourquoi le refaire apporte quelque chose de nouveau.
- `m3_credit_soc/reports/05_negative_results/main.tex` et `reports/07_income_rule/main.tex` — synthèses rédigées des résultats négatifs et du mini-protocole X1, même remarque.
- `src/m4/*.py` (dans `m4_credit_soc_fable`) — moteur actuel, fork de M3. `config.py` documente chaque champ et pourquoi ses valeurs par défaut sont ce qu'elles sont.
- **Articles et notes de conception de référence, pour les lois réelles et les pistes de mécanisme** — dossier `recherche/` (racine du dépôt `jupyter/`) :
 - `recherche/analyse_articles/` — recodage et analyse de modèles d'économie statistique : Bouchaud & Mézard, *Wealth condensation in a simple model of economy* (mécanisme de condensation de richesse par croissance multiplicative + échange, directement pertinent pour une dynamique d'accumulation-relaxation) ; deux modèles de Ian Wright sur les microfondations et l'architecture sociale d'une économie de classes. Le texte intégral des articles n'est pas versionné (copyright) mais chaque sous-dossier contient le recodage, l'analyse statistique et un rapport.
 - `recherche/analyse_distributions_taille_revenu/` — boîte à outils déjà écrite pour tester la famille de loi (13 familles, MLE, AIC/BIC) et la queue (test de Clauset–Shalizi–Newman) de distributions de taille/revenu — à réutiliser, pas à réinventer.
 - `recherche/conception_boltzmann_pareto_soc/` — note de conception antérieure à M2, qui discute déjà explicitement une transition stable $\rightarrow$ SOC pilotée par un paramètre de connectivité du marché et des diagnostics SOC (tailles de cascade, corrélation concentration/défaut). Directement pertinent pour la conception du mécanisme recherché ici.

Contexte

M3 est un modèle multi-agents de société de crédit : entités avec liquidité L et capital productif K , prêts bilatéraux, faillites en cascade, recherche d'un régime SOC (*self-organized criticality*) sur la taille des avalanches de faillites. M4 est un fork de M3. Un premier run simple

(`m4_first_s0`, seed 0, $T=2000$, baseline chocs sectoriels corrélés + objectif de revenu myope) a été exécuté et donne une population effondrée (≈ 135) et des avalanches non triviales mais petites (max= 7).

Vous travaillez dans `m4_credit_soc_fable`, un dossier de travail indépendant.

Un autre agent travaille en parallèle dans `m4_credit_soc_sol`, sur le même sujet, à partir du même point de départ. Les deux dossiers sont des copies indépendantes : ne modifiez pas `m4_credit_soc_sol`. Vous pouvez consulter le `NOTES.md` de l'autre dossier s'il existe et si cela vous aide, mais ne convergez pas prématurément vers la même solution sous ce seul prétexte — l'intérêt de deux dossiers séparés est d'obtenir deux explorations réellement indépendantes.

Avertissement fort : les résultats de M2 et de M3 sont à prendre avec des pincettes. Ils viennent d'un programme antérieur, sous des choix de conception et des critères qui ne sont pas forcément les bons pour cette phase (voir en particulier, plus bas, la correction apportée sur l'effet du réseau de crédit, qui avait été mal généralisé dans une version précédente de ce document). Traitez tout ce qui suit comme un point de départ discutable, pas comme un acquis.

Objectif

Obtenir un régime où les avalanches de faillites (composantes connexes du graphe de pertes, cf. `bankruptcy.py::_build_avalanches`) sont produites par un mécanisme d'**accumulation-relaxation endogène** — pas par amplification d'une corrélation imposée de l'extérieur — et suivent une distribution de taille en loi de puissance robuste, avec des tailles significativement au-delà de 7 (potentiellement 100+ selon la taille du système). Le système doit continuer à produire des distributions de **taille des entités** (NW, K — pas la taille des avalanches) et de **revenu** dont la **famille** de loi colle aux lois réelles étudiées dans *recherche/* (voir « Contraintes à préserver »).

Objectif final de simplification : un code simple. La liquidité d'illiquidité (défaut d'une entité solvable mais illiquide) *n'est pas un enjeu de ce programme* — ne cherchez pas à préserver ce canal pour lui-même. Simplifier les variables d'état (par exemple fusionner L et K , ou toute autre réduction) est encouragé si ça allège le code et que les contraintes ci-dessous restent respectées. En particulier, d_0 doit à terme être **complètement retiré du code** (pas seulement fixé à 0, voir plus bas) — c'est un objectif de simplification explicite, pas une option parmi d'autres.

Refonte radicale autorisée. Vous n'êtes pas limité·e à ajuster des paramètres de M4 tel quel. Toute partie du mécanisme peut être repensée si l'analyse le justifie et si le changement est documenté et comparé à la baseline actuelle : la règle de faillite (`bankruptcy.py`), la topologie de marché (`market.py`), la structure du choc, le nombre et la nature des variables d'état par entité, la règle de taux, etc. Rien n'est sanctuarisé sauf ce qui est listé plus bas comme contrainte.

Point de départ retenu pour l'investigation : la sectorialité des chocs. Commencez par l'axe des chocs sectoriels corrélés (`shock_rho_sector`, `n_sectors` — variantes G/G2 de M3), pas par la piste de dette de subsistance (qui reste une voie possible mais pas privilégiée, voir plus bas). C'est un choix délibéré : M3 a lu cet axe comme une amplification continue sans transition, mais ce jugement reposait notamment sur le critère var/mean, dont ce document explique plus bas pourquoi il n'est pas nécessaire — l'axe mérite d'être ré-examiné avec le bon critère.

Marqueur déjà observé, à prendre au sérieux : régression en loi de puissance après exclusion des avalanches de taille 1. En excluant les avalanches de taille 1 de la régression log-log (garder les deux régressions, avec et sans taille 1, affichées), on obtient déjà de très bons

ajustements ($r^2 > 0,90$) sur des runs existants. C’est un très bon marqueur d’un régime auto-critique — bien meilleur que var/mean — à calculer systématiquement pour chaque mécanisme candidat (voir « Critères de succès »).

Ce que M3 a établi (points de départ, à ré-examiner, pas des vérités intangibles)

1. **Corrélation imposée vs auto-organisation — axe de départ de l’investigation.** G1 (choc macro) et G2/G2b/G2c (choc sectoriel, ρ_s de 0 à 0,8) montraient dans M3 une réponse jugée *continue* : var/mean des avalanches de 0,07 (iid) à 0,84 ($\rho_s=0,8$), max jusqu’à 72, sans transition nette au sens de ce critère. Mais var/mean n’est pas le bon critère (voir « Critères de succès ») — c’est précisément pourquoi cet axe est repris en premier ici, avec le marqueur de régression hors taille 1 comme signal principal.
2. **Décision de conception retenue pour M4 : d_0 est supprimé.** Le run H de M3 ($d_0=0$) a montré que le système se borne seul par l’insolvabilité contractuelle (population stationnaire, renouvellement observé) sans plancher exogène : on sait donc qu’on peut s’en passer. Commencez par $d_0=0$ en configuration si besoin d’une étape intermédiaire, mais l’objectif final est de retirer d_0 et son usage **du code lui-même** (`config.py`, `bankruptcy.py:: net_worth`), pas seulement de le neutraliser par sa valeur. La piste consistant à router une dette de subsistance vers de vraies créancières du marché (plutôt que vers un puits abstrait) reste une voie possible à explorer, mais ce n’est **pas la seule ni la prioritaire** — l’axe retenu en premier est la sectorialité des chocs (ci-dessus).
3. **Le réseau de crédit a un effet réel et attendu sur les cascades.** Activer le crédit (vs l’ablation B, sans marché) multiplie les cascades : c’est le canal même qui les rend possibles en connectant les entités entre elles, et c’est parfaitement normal — ne traitez aucun résultat de ce document comme disant le contraire. Ce que la dose-réponse F/F'' /baseline de M3 a réellement fermé est plus étroit : à *volume de crédit égal*, changer la règle d’appariement (assortative / random / random_lender) ne changeait rien de mesuré au-delà de ce volume — mais ce résultat lui-même est à ré-examiner avec les bons critères plutôt qu’à prendre pour acquis.
4. **Le crédit est causal démographiquement, pas distributionnellement**, en baseline richesse (A vs B) selon M3 ; ce n’est qu’avec la règle de revenu (X1) que le crédit devenait distributionnellement actif (Gini 0,62–0,69 vs 0,56). À vérifier plutôt qu’à supposer si votre mécanisme candidat change la donne.

Contraintes à préserver

- **Familles de distribution.** Corps de NW, K , revenu doivent rester dans des familles reconnaissables (pas nécessairement identiques à Fisk/dPIN/lognormal observées en M3 — à re-tester, pas à supposer). Pour les lois réelles de référence, voir `recherche/analyse_articles/` et `recherche/analyse_distributions_taille_revenu/` (lecture préalable ci-dessus) plutôt que de partir de zéro.
- **Renouvellement démographique minimal, pas un taux de mortalité cible.** La seule exigence est qu’il existe un renouvellement (turnover) de la population/du haut de la distribution, même très lent — pas de taux de mortalité particulier à atteindre, et ne réutilisez pas un critère de mortalité instantanée du top (voir pièges d’outillage) : mesurez le renouvellement entre fenêtres temporelles.
- **Invariants comptables du moteur** (cascade de faillite avec point fixe garanti, snapshot en lecture pure pour les figures, etc. — voir docstrings `bankruptcy.py`, `contracts.py`, `tests tests/`). Toute refonte du moteur doit préserver un jeu d’invariants équivalent et le tester.

- **Reproductibilité** : `random.Random(seed)` isolé, jamais le RNG global ; artefacts sur disque (`config.json`, `summary.json`, `series.csv`, `avalanches.csv`, `snap_t*.npz`) comme seule source pour les figures — jamais l'état mémoire de la simulation.

Pièges d'outillage à ne pas reproduire

1. Test LR Pareto/lognormale **sans renormalisation** de la lognormale tronquée → biais pro-Pareto systématique.
2. AIC du corps de distribution avec des familles **non tronquées** sur un corps qui est en réalité tronqué → biais anti-exponentiel.
3. Transfert prorata des créances de faillite sans fusion par paire (`merge_pairs`) → explosion combinatoire du carnet de prêts (observé jusqu'à 12,4 M de contrats sur la grille M3).
4. Moyenne arithmétique des taux d'intérêt au lieu de géométrique → le marché de crédit s'éteint de lui-même.
5. Critère de « mortalité instantanée du top » — mauvais critère, structurellement nul. Ce qu'on veut n'est qu'un **renouvellement**, même très lent : mesurez-le entre fenêtres temporelles, pas instantanément.
6. **Période transitoire**. Ne calculez pas de statistiques sur des données qui incluent encore le régime transitoire de démarrage — identifiez et excluez le burn-in avant d'ajuster une distribution ou de compter des avalanches.
7. **Robustesse de la fenêtre d'analyse**. Faites varier le temps de simulation T pour vérifier que le choix de fenêtre d'analyse (où commence le régime stationnaire, sur quelle plage on ajuste) est robuste, pas un artefact d'un T particulier.
8. **Variance inter-seeds**. Pour chaque point de paramètres, utilisez plusieurs seeds afin d'estimer la variance des résultats — un seed unique ne suffit pas pour juger qu'un effet est réel.

Critères de succès

1. **Marqueur principal : régression en loi de puissance après exclusion des avalanches de taille 1**, $r^2 > 0,90$. Gardez les deux régressions affichées (toutes tailles, et hors taille 1) — c'est le signal le plus net déjà observé d'un régime auto-critique. À calculer systématiquement pour chaque mécanisme candidat.
2. **Scaling en taille finie** : le cutoff de la distribution de taille d'avalanches croît avec la taille du système (population stationnaire ou nombre d'entités actives), signature classique d'un régime critique.
3. **Ajustement en loi de puissance qui survit au test LR corrigé** (avec la renormalisation correcte des alternatives tronquées — piège n°1 ci-dessus), pas seulement une droite à l'œil sur un graphe log-log.
4. **La condition sur-Poisson ($\text{var}/\text{mean} > 1$) N'EST PAS nécessaire**. Ne l'utilisez pas comme critère de rejet d'un mécanisme — M3 en a fait un usage excessif alors que ce n'est pas ce qui compte (voir le marqueur principal ci-dessus).
5. Les distributions de taille des entités et de revenu gardent une famille reconnaissable, en cohérence avec les lois réelles de référence (**recherche/**), avec le changement documenté si une famille change.
6. Le mécanisme proposé a une **histoire causale endogène** articulable en une phrase (pas juste « on a poussé un paramètre de corrélation plus haut ») — feedback d'accumulation puis de relaxation porté par la dynamique du modèle lui-même.

7. Piège à nommer explicitement : un effondrement quasi total n’est pas de la SOC.

La façon la moins chère d’obtenir de « grosses avalanches » dans un modèle de cascade de levier est un régime bimodal (une masse de défauts de taille 1 + un effondrement occasionnel de tout le système) — ce n’est ni une loi de puissance ni un régime critique, c’est une extinction. Un run qui rapporte « avalanche de 100+ obtenue ! » doit être vérifié contre ce scénario (et contre les critères 1–3) avant d’être compté comme un succès.

Livrables attendus

- Modifications du moteur dans `src/m4/` (ou nouveau module si la refonte le justifie), avec tests couvrant les invariants touchés.
- Au moins un run comparatif baseline-vs-mécanisme-candidat avec figures (réutiliser les recettes déjà établies : `simulation_lab/plot_utils.py` et `webapp/mpl_figures.py` — log-binning adaptatif, erreur bootstrap, double régression avec/sans taille 1, écrites pour `cascades_rank_size.py` dans `m3_credit_soc` — à copier/adapter, pas à réinventer).
- Une entrée de journal dans `m4_credit_soc_fable/NOTES.md` (sur le modèle de celui de M3 : un résumé d’une ligne en tête de chaque entrée, confirmé ou infirmé, et pourquoi) pour chaque hypothèse testée, y compris les échecs — les négatifs sont aussi précieux que les positifs dans ce programme.

Quand s’arrêter et demander

Ne vous arrêtez pas pour demander confirmation. Continuez le travail de bout en bout, y compris pour les décisions notables (changer une convention documentée ailleurs comme figée, lancer un run coûteux, conclure qu’aucun mécanisme testé ne produit de SOC robuste). Dans tous ces cas, **notifiez-le dans le journal / le rapport** — ce qui compte est que la décision et sa justification soient tracées, pas qu’elle soit validée avant d’être prise.

Instructions d’exécution — Claude Fable 5

Vous êtes l’agent de développement chargé de cette phase de recherche. Quelques attentes propres à la façon dont vous travaillez sur des tâches longues et ouvertes :

- **Mémoire de travail.** Tenez `m4_credit_soc/NOTES.md` comme journal au fil de l’eau, une entrée par hypothèse testée (cf. « Livrables attendus ») : résumé d’une ligne en tête, confirmé/infirmé, et pourquoi. Ne dupliquez pas ce que l’historique git ou le code documente déjà.
- **Vérification avant de rapporter.** Avant d’annoncer un résultat comme acquis, auditez chaque affirmation contre une sortie d’outil réelle de cette session (run, test, figure). Si quelque chose n’est pas vérifié, dites-le explicitement plutôt que de l’affirmer.
- **Subagents pour la vérification longue.** Pour les runs longs ou les ajustements statistiques (tests LR, scaling en taille finie), préférez un subagent à contexte frais pour re-vérifier plutôt que l’auto-critique seule.
- **Ambition.** Ce sujet est délibérément ouvert (refonte radicale autorisée) — c’est le type de tâche où vous performez le mieux. N’hésitez pas à proposer et comparer plusieurs mécanismes candidats plutôt que de vous limiter au premier qui fonctionne un peu.
- **Autonomie.** Ne vous arrêtez pas pour demander confirmation, même sur les décisions notables — notifiez-les dans le journal (voir « Quand s’arrêter et demander » ci-dessus) et continuez. Ne vous arrêtez que sur une action destructive/irréversible/hors-périmètre.

- **Dossier de travail indépendant.** Travaillez exclusivement dans `m4_credit_soc_fable`. Ne modifiez pas `m4_credit_soc_sol` (l'autre agent y travaille en parallèle sur le même sujet, indépendamment).